

Day 7 Assignment

Securing and Documenting a REST API with OAuth2, JWT and Rate Limiting

John Michael Bilbao
Techstart
August 18, 2026

Introduction

This assignment asks for an API secured with OAuth2 and JWT, protected with rate limiting, documented with Swagger and Postman, and put to use by a practical client.

What I built is an expense reports API. The reports themselves are incidental; they exist so there is something worth protecting. Three accounts have different permissions, which is what makes it possible to show the difference between a caller the API does not recognise, one it recognises but will not let through, and one it lets through.

The API is FastAPI. I chose it over Flask for this particular assignment because its OAuth2PasswordBearer is a real implementation of the password grant rather than a bearer-token helper: it produces the Authorize dialog in Swagger UI with the scopes as checkboxes, and that dialog performs an actual token exchange. Every screenshot of a refusal in this document is a response the running API sent.

Everything is run from PowerShell on Windows.

Step 1: The API

Nine endpoints. Only /health is open; everything else needs a token, and the reports endpoints each need a specific scope.

Endpoint	Purpose	Scope required
POST /auth/token	Exchange username and password for tokens	— open
POST /auth/refresh	Trade a refresh token for a new access token	a refresh token
GET /auth/me	Who the presented token belongs to	any token
GET /reports	List reports	reports:read
GET /reports/{id}	One report	reports:read
GET /reports/summary	Totals by category	reports:read
POST /reports	Create a report	reports:write
DELETE /reports/{id}	Delete a report	reports:delete

Endpoint	Purpose	Scope required
GET /health	Service status	— open

The three accounts, and what each may be granted:

Account	Password	Scopes it may hold
analyst	analyst-password	reports:read
manager	manager-password	reports:read, reports:write
admin	admin-password	reports:read, reports:write, reports:delete

There is also a disabled account, so the difference between credentials being wrong and an account being turned off can be shown.

Setting up is four packages, plus Newman for running the Postman collection from the command line:

```

>_ Windows PowerShell

PS C:\Users\John\secure-api> python -m venv .venv
PS C:\Users\John\secure-api> .\.venv\Scripts\Activate.ps1
PS C:\Users\John\secure-api> pip install -r requirements.txt
Successfully installed fastapi-0.128.8 pyjwt-2.13.0 bcrypt-5.0.0 slowapi-0.1.9
PS C:\Users\John\secure-api> npm install -g newman
added 137 packages in 6s

PS C:\Users\John\secure-api> python -c "import fastapi, jwt, bcrypt, slowapi; print(...)"
Python          3.9.25
fastapi         0.128.8
pyjwt           2.13.0
bcrypt          5.0.0
Traceback (most recent call last):
  File "<string>", line 1, in <module>
AttributeError: module 'slowapi' has no attribute '__version__'

```

Figure 1: The environment and the versions in it

The FastAPI command line tool runs it, and reports where the documentation is:

```
>_ Windows PowerShell - fastapi dev
PS C:\Users\John\secure-api> fastapi dev secure_api\main.py

FastAPI   Starting development server

        Searching for package file structure from directories with
        __init__.py files
        Importing from C:\Users\John\secure-api

module    [] secure_api
           |— [] __init__.py
           |— [] main.py

code      Importing the FastAPI app object from the module with the following
           code:

           from secure_api.main import app

app       Using import string: secure_api.main:app

server    Server started at http://127.0.0.1:8001
server    Documentation at http://127.0.0.1:8001/docs

tip       Running in development mode, for production use: fastapi run

Logs:

INFO      Will watch for changes in these directories:
           ['C:\Users\John\secure-api']
INFO      Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO      Started reloader process [8] using WatchFiles
INFO      Started server process [11]
INFO      Waiting for application startup.
INFO      Application startup complete.
```

Figure 2: fastapi dev starting the server

The open endpoint answers without any credentials, and says which scopes exist:

```
> curl - open endpoint

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 136
content-type: application/json

{
  "status": "ok",
  "authentication": "OAuth2 password flow, JWT bearer",
  "scopes_available": [
    "reports:delete",
    "reports:read",
    "reports:write"
  ]
}
```

Figure 3: GET /health, no token needed

Step 2: OAuth2 and JWT

OAuth2 is a framework of several flows rather than one protocol. The one used here is the password grant, which is the flow for a client the API owner controls: the client collects the username and password, posts them once to the token endpoint, and from then on holds a token instead. The credentials are not stored by the client and not sent again.

The token endpoint takes form fields rather than JSON, which is what OAuth2 specifies. It returns two tokens:

```
> curl - POST /auth/token

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/auth/token `
-H "Content-Type: application/x-www-form-urlencoded" `
-d "username=manager&password=manager-password"
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 771
content-type: application/json

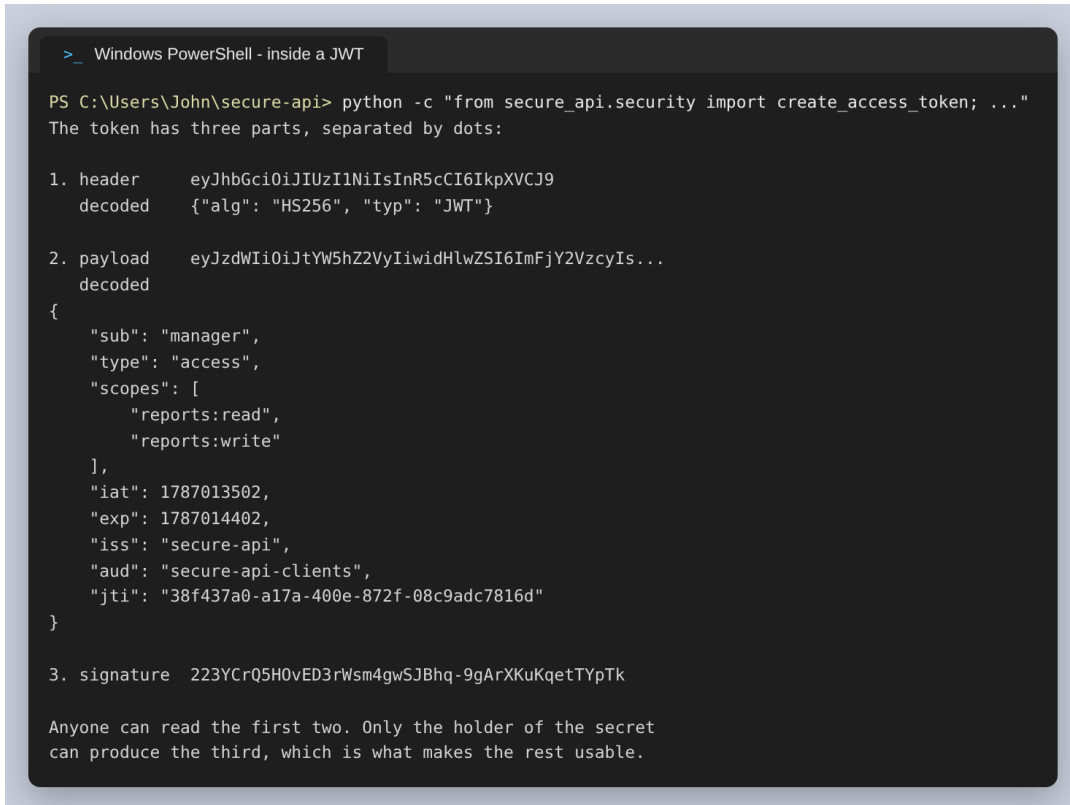
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtYW5h...kZGVlIn0.un_jAWHkqI...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtYW5h...OTJmZSJ9.OHqTvZsbRk...",
  "token_type": "bearer",
  "expires_in": 900,
  "scope": "reports:read reports:write"
}
```

Figure 4: POST /auth/token. The tokens are shortened in this screenshot; a real one is about 300 characters.

Two tokens rather than one is the part worth explaining. The access token expires in 15 minutes, which limits the damage if it leaks, but nobody wants to retype a password every 15 minutes. The refresh token lasts 24 hours and can be exchanged for a new access token. It carries no scopes at all, so it cannot be used to reach a protected endpoint even though it is a perfectly valid JWT.

What is actually inside a JWT

A JWT is three base64 sections joined by dots. It is worth being precise about what that means: the first two are encoded, not encrypted. Anyone holding the token can read them.



```
>_ Windows PowerShell - inside a JWT

PS C:\Users\John\secure-api> python -c "from secure_api.security import create_access_token; ..."
The token has three parts, separated by dots:

1. header      eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
   decoded     {"alg": "HS256", "typ": "JWT"}

2. payload     eyJzdWIiOiJtYW5hZ2VyaWwidHlwZSI6ImFjY2VzcyIs...
   decoded     {
     "sub": "manager",
     "type": "access",
     "scopes": [
       "reports:read",
       "reports:write"
     ],
     "iat": 1787013502,
     "exp": 1787014402,
     "iss": "secure-api",
     "aud": "secure-api-clients",
     "jti": "38f437a0-a17a-400e-872f-08c9adc7816d"
   }

3. signature    223YCrQ5H0vED3rWsm4gwSJBhq-9gArXKuKqetTYpTk

Anyone can read the first two. Only the holder of the secret
can produce the third, which is what makes the rest usable.
```

Figure 5: A real token taken apart. The payload is readable by anyone who has the token.

So the security does not come from hiding the claims. It comes from the third section, the signature. The API signs the header and payload with a secret using HS256; anyone can verify the token is intact, but only the holder of the secret can produce a signature for claims they have changed. That is why nothing secret is ever put in a payload.

The claims are what the API relies on. `sub` is the account, `scopes` is what the token may do, `exp` is when it stops working, `iat` is when it was issued, `iss` and `aud` say who issued it and who it was issued for, and `jti` is a unique id for this token. This is the function that assembles them:

```
• security.py  dependencies.py  limits.py

53 def _encode(subject: str, token_type: str, lifetime: timedelta,
54             scopes: Iterable[str] = ()) -> str:
55     now = datetime.now(timezone.utc)
56     payload = {
57         "sub": subject,
58         "type": token_type,
59         "scopes": list(scopes),
60         "iat": int(now.timestamp()),
61         "exp": int((now + lifetime).timestamp()),
62         "iss": ISSUER,
63         "aud": AUDIENCE,
64         # A unique id per token, so a specific one could be revoked later.
65         "jti": str(uuid.uuid4()),
66     }
67     return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
68
69
70 def create_access_token(subject: str, scopes: Iterable[str],
71                        minutes: Optional[int] = None) -> str:
72     return _encode(subject, "access",
73                   timedelta(minutes=minutes if minutes is not None
74                             else ACCESS_TOKEN_MINUTES),
75                   scopes)
```

secure_api/security.py Python Spaces: 4 UTF-8

Figure 6: Building the payload and signing it

Verifying is where the care goes. Every claim that was written has to be checked, and each failure has to produce a message that helps a legitimate client without helping an attacker:

```

• security.py  dependencies.py  limits.py

83 def decode_token(token: str, expected_type: str = "access") -> dict:
84     """Verify a token and return its payload.
85
86     Every failure raises TokenError with a message safe to send to the client.
87     The signature, the expiry, the issuer and the audience are all checked, and
88     then the type, so an access token and a refresh token are not
89     interchangeable.
90     """
91     try:
92         payload = jwt.decode(
93             token, SECRET_KEY, algorithms=[ALGORITHM],
94             audience=AUDIENCE, issuer=ISSUER,
95             options={"require": ["exp", "iat", "sub", "type"]},
96         )
97     except jwt.ExpiredSignatureError:
98         raise TokenError("The token has expired.")
99     except jwt.InvalidAudienceError:
100         raise TokenError("The token was not issued for this API.")
101     except jwt.InvalidIssuerError:
102         raise TokenError("The token was not issued by this API.")
103     except jwt.InvalidSignatureError:
104         raise TokenError("The token signature does not match.")
105     except jwt.InvalidTokenError as exc:
106         raise TokenError(f"The token could not be read: {exc}")
107
108     if payload.get("type") != expected_type:
109         raise TokenError(
110             f"Expected a {expected_type} token, got a "
111             f"{payload.get('type')} token.")
112
113     return payload

```

secure_api/security.py Python Spaces: 4 UTF-8

Figure 7: Verifying a token. The type check is what stops a refresh token being used as an access token.

Passwords are never stored, only bcrypt hashes. bcrypt is deliberately slow and salts each hash, so two accounts with the same password get different hashes and a stolen list cannot be reversed with a lookup table:

```

>_ Windows PowerShell - password storage

PS C:\Users\John\secure-api> python -c "from secure_api.users import USERS; ..."
stored for manager:
    $2b$12$dhCsP27BwU6wZn5TfXVwcuSaj.4MDWil0nWrNurjVDijf7J8So75a

algorithm      2b
cost factor    12
length         60 characters

correct password -> True
wrong password  -> False

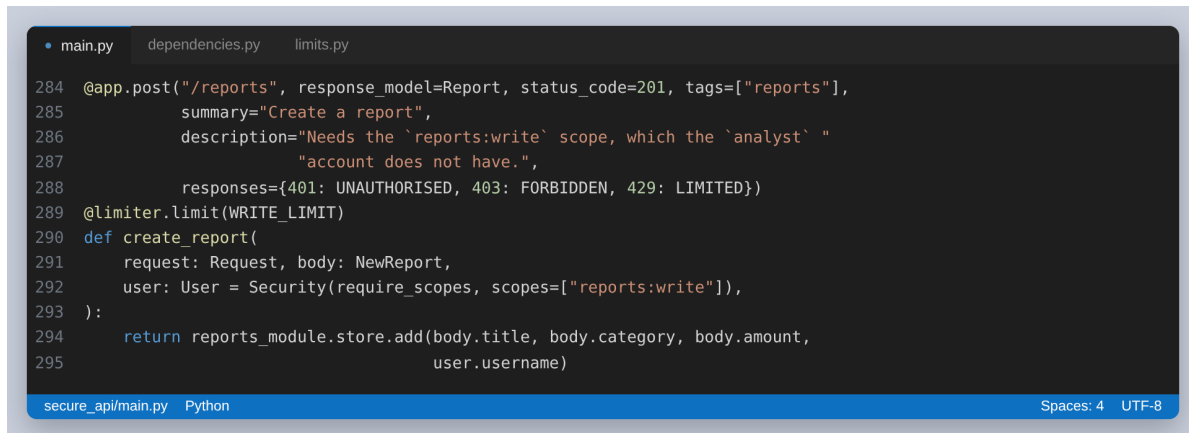
```

Figure 8: What is stored for the manager account: algorithm 2b, cost factor 12, 60 characters

Authentication is not authorisation

This is the distinction the scopes exist to enforce, and the one the status codes have to reflect. A 401 means the API does not know who you are. A 403 means it knows exactly who you are and is refusing anyway. Returning 401 for both would tell a client to sign in again when signing in again would not help.

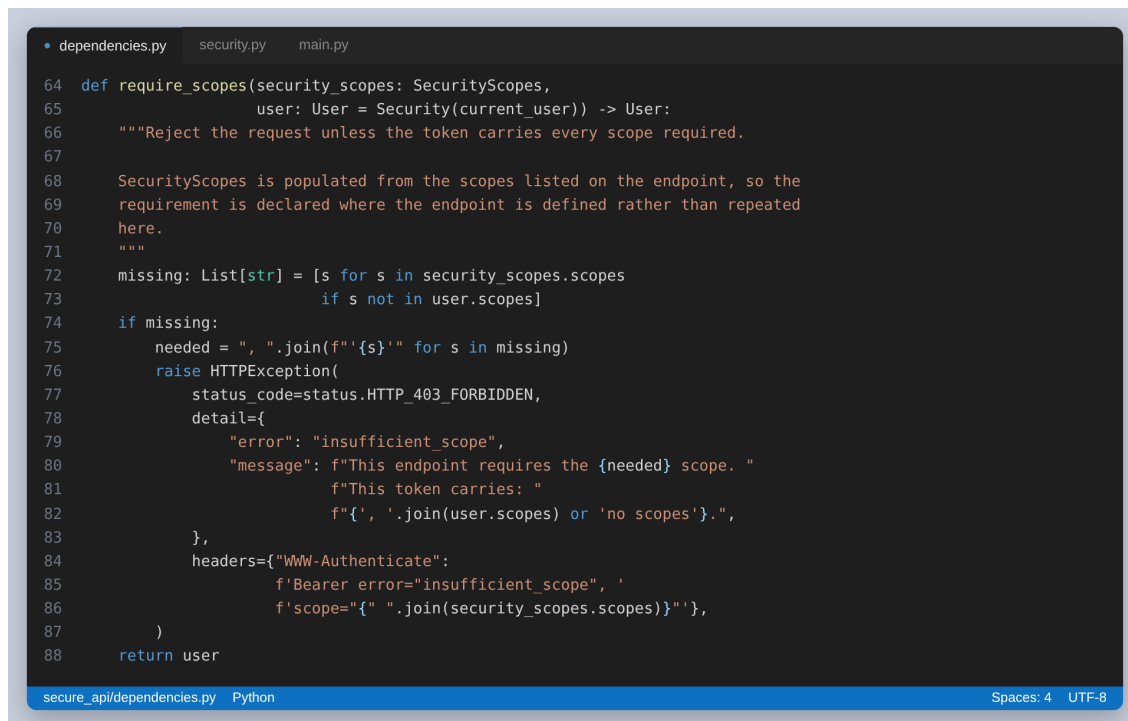
FastAPI's SecurityScopes makes the requirement part of the endpoint declaration, so the rule lives where the endpoint is defined:



```
• main.py dependencies.py limits.py
284 @app.post("/reports", response_model=Report, status_code=201, tags=["reports"],
285         summary="Create a report",
286         description="Needs the `reports:write` scope, which the `analyst` "
287             "account does not have.",
288         responses={401: UNAUTHORISED, 403: FORBIDDEN, 429: LIMITED})
289 @limiter.limit(WRITE_LIMIT)
290 def create_report(
291     request: Request, body: NewReport,
292     user: User = Security(require_scopes, scopes=["reports:write"]),
293 ):
294     return reports_module.store.add(body.title, body.category, body.amount,
295                                     user.username)
secure_api/main.py Python Spaces: 4 UTF-8
```

Figure 9: The scope required is declared on the endpoint itself

And the dependency that enforces it:



```
• dependencies.py security.py main.py
64 def require_scopes(security_scopes: SecurityScopes,
65                     user: User = Security(current_user)) -> User:
66     """Reject the request unless the token carries every scope required.
67
68     SecurityScopes is populated from the scopes listed on the endpoint, so the
69     requirement is declared where the endpoint is defined rather than repeated
70     here.
71     """
72     missing: List[str] = [s for s in security_scopes.scopes
73                           if s not in user.scopes]
74     if missing:
75         needed = ", ".join(f'{s}' for s in missing)
76         raise HTTPException(
77             status_code=status.HTTP_403_FORBIDDEN,
78             detail={
79                 "error": "insufficient_scope",
80                 "message": f"This endpoint requires the {needed} scope. "
81                     f"This token carries: "
82                     f'{' '.join(user.scopes) or 'no scopes'}.',
83             },
84             headers={"WWW-Authenticate":
85                     f'Bearer error="insufficient_scope", '
86                     f'scope="{ " ".join(security_scopes.scopes)}"'},
87         )
88     return user
secure_api/dependencies.py Python Spaces: 4 UTF-8
```

Figure 10: Refusing a request whose token lacks a scope, with the scope named in the response

A token also carries only what was asked for. The admin account may hold all three scopes, but a client can request fewer, and then the token can do less. A client that only reads should ask only to read:

```
> _ curl - asking for fewer scopes

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/auth/token `
  -H "Content-Type: application/x-www-form-urlencoded" `
  -d "username=admin&password=admin-password&scope=reports:read"
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:53 GMT
server: uvicorn
content-length: 731
content-type: application/json

{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1p...2ZTE0In0.GJwjc18QDx...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1p...VkZGYifQ.jH1Rptv5xX...",
  "token_type": "bearer",
  "expires_in": 900,
  "scope": "reports:read"
}
```

Figure 11: The admin account signing in with reports:read only

Reading back what a token represents:

```
> _ curl - GET /auth/me

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/auth/me -H "Authorization: Bearer $env:TOKEN"
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 247
content-type: application/json

{
  "username": "manager",
  "full_name": "Man Ager",
  "role": "manager",
  "scopes": [
    "reports:read",
    "reports:write"
  ],
  "token_issued_at": "2026-08-18T00:40:52+00:00",
  "token_expires_at": "2026-08-18T00:55:52+00:00",
  "token_id": "96f72383-0f0f-4f30-88ac-288338f43037"
}
```

Figure 12: GET /auth/me

Listing the protected data:

```
> _ curl - GET /reports

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports -H "Authorization: Bearer $env:TOKEN"
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 996
content-type: application/json

{
  "count": 6,
  "total_amount": 40849.0,
  "reports": [
    {
      "id": 1,
      "title": "Client visit, Cebu",
      "category": "travel",
      "amount": 12450.0,
      "submitted_by": "manager",
      "status": "approved",
      "created_at": "2026-08-18T00:40:51+00:00"
    },
    {
      "id": 2,
      "title": "Team lunch, Makati",
      "category": "meals",
      "amount": 3200.0,
      "submitted_by": "manager",
      "status": "approved",
      "created_at": "2026-08-18T00:40:51+00:00"
    },
    "... 4 more not shown"
  ]
}
```

Figure 13: GET /reports with a valid token. The list is shortened here.

The aggregate endpoint:

```

> _ curl - GET /reports/summary

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports/summary -H "Authorization: Bearer $env:TOKEN"
HTTP/1.1 200 OK
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 298
content-type: application/json

{
  "report_count": 6,
  "total_amount": 40849.0,
  "by_category": {
    "travel": 13300.0,
    "meals": 3200.0,
    "equipment": 6349.0,
    "training": 18000.0
  },
  "largest": {
    "id": 4,
    "title": "Conference ticket",
    "category": "training",
    "amount": 18000.0,
    "submitted_by": "analyst",
    "status": "pending",
    "created_at": "2026-08-18T00:40:51+00:00"
  }
}

```

Figure 14: GET /reports/summary

A write, which needs a second scope:

```

> _ curl - POST /reports

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/reports `
-H "Authorization: Bearer $env:TOKEN" `
-H "Content-Type: application/json" `
-d '{"title": "Client workshop, Iloilo", "category": "travel", "amount": 9800}'
HTTP/1.1 201 Created
date: Tue, 18 Aug 2026 00:40:52 GMT
server: uvicorn
content-length: 163
content-type: application/json

{
  "id": 7,
  "title": "Client workshop, Iloilo",
  "category": "travel",
  "amount": 9800.0,
  "submitted_by": "manager",
  "status": "pending",
  "created_at": "2026-08-18T00:40:52+00:00"
}

```

Figure 15: POST /reports as manager

And a delete, which only admin can do:

```
>_ curl - DELETE as admin

PS C:\Users\John\secure-api> curl.exe -i -X DELETE http://127.0.0.1:8000/reports/2 `
-H "Authorization: Bearer $env:ADMIN_TOKEN"
HTTP/1.1 204 No Content
date: Tue, 18 Aug 2026 00:40:53 GMT
server: uvicorn
content-type: application/json
```

Figure 16: DELETE /reports/2 as admin

Every way a request gets turned away

This is the part I spent most of the time on, because an API is only as good as its refusals. No token at all:

```
>_ curl - 401, no token

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports
HTTP/1.1 401 Unauthorized
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 47
content-type: application/json

{
  "error": "error",
  "message": "Not authenticated"
}
```

Figure 17: 401 with a WWW-Authenticate header

A token with one character of its signature changed. The claims are still perfectly readable, and it still does not work, which is the whole point of signing:

```
>_ curl - 401, edited signature

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports -H "Authorization: Bearer $env:TAMPERED"
HTTP/1.1 401 Unauthorized
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer error="invalid_token"
content-length: 73
content-type: application/json

{
  "error": "invalid_token",
  "message": "The token signature does not match."
}
```

Figure 18: 401, the signature does not match

A token that has expired:

```
> curl - 401, expired token

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports -H "Authorization: Bearer $env:EXPIRED"
HTTP/1.1 401 Unauthorized
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer error="invalid_token"
content-length: 60
content-type: application/json

{
  "error": "invalid_token",
  "message": "The token has expired."
}
```

Figure 19: 401, expired

A refresh token presented where an access token belongs. Both are valid JWTs signed by this API; the type claim is what separates them:

```
> curl - 401, refresh token used for access

PS C:\Users\John\secure-api> curl.exe -i http://127.0.0.1:8000/reports -H "Authorization: Bearer $env:REFRESH"
HTTP/1.1 401 Unauthorized
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer error="invalid_token"
content-length: 83
content-type: application/json

{
  "error": "invalid_token",
  "message": "Expected a access token, got a refresh token."
}
```

Figure 20: 401, the wrong kind of token

A valid token whose scopes do not stretch far enough. Note the 403 and that the message names both the scope needed and the scopes held:

```
> _ curl - 403, scope missing

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/reports `
-H "Authorization: Bearer $env:ANALYST_TOKEN" `
-H "Content-Type: application/json" `
-d '{"title": "Not allowed", "category": "meals", "amount": 100}'
HTTP/1.1 403 Forbidden
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer error="insufficient_scope", scope="reports:write"
content-length: 126
content-type: application/json

{
  "error": "insufficient_scope",
  "message": "This endpoint requires the 'reports:write' scope. This token carries: reports:read."
}
```

Figure 21: 403 for the analyst account attempting a write

The same for a manager attempting a delete, which shows that write is not a superset of delete:

```
> _ curl - 403, write is not delete

PS C:\Users\John\secure-api> curl.exe -i -X DELETE http://127.0.0.1:8000/reports/2 `
-H "Authorization: Bearer $env:TOKEN"
HTTP/1.1 403 Forbidden
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer error="insufficient_scope", scope="reports:delete"
content-length: 142
content-type: application/json

{
  "error": "insufficient_scope",
  "message": "This endpoint requires the 'reports:delete' scope. This token carries: reports:read, reports:write."
}
```

Figure 22: 403, reports:write is not reports:delete

A wrong password. The message is deliberately the same one an unknown username produces, because a different message would let someone work out which accounts exist:

```
> curl - 401, wrong password

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/auth/token `
-H "Content-Type: application/x-www-form-urlencoded" `
-d "username=manager&password=not-the-password"
HTTP/1.1 401 Unauthorized
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 75
content-type: application/json

{
  "error": "invalid_credentials",
  "message": "Incorrect username or password."
}
```

Figure 23: 401, wrong password

And asking for a scope the account may not hold, which is refused rather than quietly granted in a reduced form:

```
> curl - 400, scope refused

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/auth/token `
-H "Content-Type: application/x-www-form-urlencoded" `
-d "username=analyst&password=analyst-password&scope=reports:delete"
HTTP/1.1 400 Bad Request
date: Tue, 18 Aug 2026 00:40:55 GMT
server: uvicorn
content-length: 113
content-type: application/json

{
  "error": "invalid_scope",
  "message": "This account cannot be granted 'reports:delete'. It may have: reports:read."
}
```

Figure 24: 400, scope refused

Step 3: Rate limiting

Rate limiting is here because authentication alone does not stop abuse. Nothing in the token logic prevents someone posting thousands of password guesses to the token endpoint, and each one would be correctly rejected while the attacker worked through a word list.

SlowAPI is the FastAPI equivalent of Flask-Limiter and works the same way: a limit per endpoint, counted against a key. Four limits are used:

Endpoint	Limit	Counted against	Why
POST /auth/token	5 / minute	the client address	This is where password guessing happens
POST, DELETE /reports	10 / minute	the token id	Writes cost more than reads
GET /reports/summary	20 / minute	the token id	Aggregates more work per call
Everything else	60 / minute	the token id	A sane default

The key matters as much as the number. Counting reads per address would mean an office behind one address shares a single allowance, and one busy client could lock out its colleagues. So an authenticated request is counted against the token's jti claim instead. Logins have no token yet, so those are counted per address, which is the right key for the thing being defended against:

```

• limits.py  main.py  security.py

25 def client_key(request: Request) -> str:
26     """Who to count against.
27
28     The address on its own is a poor key when clients share one, so an
29     authenticated request is counted against its token id instead. That also
30     stops one caller's traffic from using up another's allowance.
31     """
32     auth = request.headers.get("authorization", "")
33     if auth.lower().startswith("bearer "):
34         token = auth.split(" ", 1)[1]
35         # Read the id without verifying: this is only a bucket key, and an
36         # invalid token is rejected by the dependency regardless.
37         try:
38             import jwt
39             claims = jwt.decode(token, options={"verify_signature": False})
40             if claims.get("jti"):
41                 return f"token:{claims['jti']}"
42         except Exception:
43             pass
44     return f"ip:{get_remote_address(request)}"
45
46
47 def login_key(request: Request) -> str:
48     """Count login attempts per address, so one client cannot spread them."""
49     return f"login:{get_remote_address(request)}"

```

Figure 25: Choosing what to count a request against

Six password guesses in a row. The first five are refused as wrong; the sixth is refused for being the sixth. The call after that has the correct password and is still refused, which is what makes this useful:

```

> curl - 429, too many attempts

PS C:\Users\John\secure-api> 1..6 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code}`n" `
    -X POST http://127.0.0.1:8000/auth/token `
    -d "username=manager&password=guess" }
attempt 1: HTTP 401
attempt 2: HTTP 401
attempt 3: HTTP 401
attempt 4: HTTP 401
attempt 5: HTTP 401
attempt 6: HTTP 429

PS C:\Users\John\secure-api> curl.exe -i -X POST http://127.0.0.1:8000/auth/token `
    -H "Content-Type: application/x-www-form-urlencoded" `
    -d "username=manager&password=manager-password"
HTTP/1.1 429 Too Many Requests
date: Tue, 18 Aug 2026 00:40:59 GMT
server: uvicorn
retry-after: 60
content-length: 145
content-type: application/json

{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Slow down and try again shortly.",
  "limit": "5 per 1 minute",
  "retry_after_seconds": 60
}

```

Figure 26: 429 after five attempts, with the limit and a Retry-After header

The 429 says what the limit was and when to try again, so a legitimate client that has been too eager can back off correctly instead of guessing.

Step 4: Documentation, with Swagger

FastAPI generates the OpenAPI document from the type hints and the response models. Nothing in the pages below was written separately; the descriptions come from the endpoint decorators and the Pydantic fields.

Secure Expense Reports API

1.0.0 OAS 3.1

/openapi.json

An expense reports API, used here to demonstrate how the endpoints are secured rather than for the reports themselves.

Signing in

POST /auth/token takes a username and password as form fields, the OAuth2 password grant, and returns a short lived **access token** plus a longer lived **refresh token**. Send the access token as **Authorization: Bearer <token>**.

Press **Authorize** to sign in from this page. The scopes are checkboxes: a token only carries what was asked for, so requesting fewer scopes gives a token that can do less.

Accounts

Username	Password	Scopes
analyst	analyst-password	reports:read
manager	manager-password	reports:read , reports:write
admin	admin-password	all three

Rate limits

5/minute on the token endpoint, counted per address, because that is the endpoint a password guesser would use. **10/minute** on writes, **20/minute** on the summary and **60/minute** everywhere else, counted per token. Going over returns **429** with a **Retry-After** header.

Figure 27: The description at /docs, including the accounts and the rate limits

The endpoints, each with a padlock showing it is protected:

auth Getting and refreshing tokens.

POST	/auth/token	Exchange username and password for tokens	✓
POST	/auth/refresh	Trade a refresh token for a new access token	🔒 ✓
GET	/auth/me	Who this token belongs to	🔒 ✓

reports The protected data. Every endpoint needs a scope.

GET	/reports/summary	Totals by category	🔒 ✓
GET	/reports	List reports	🔒 ✓
POST	/reports	Create a report	🔒 ✓
GET	/reports/{report_id}	One report by id	🔒 ✓
DELETE	/reports/{report_id}	Delete a report	🔒 ✓

service Open endpoints.

GET	/health	Open endpoint, no token needed	✓
-----	---------	--------------------------------	---

Figure 28: All nine endpoints. Only /health has no padlock.

The Authorize button opens the OAuth2 dialog. This is the clearest evidence that the flow is real OAuth2 rather than a bearer token pasted into a header: Swagger UI knows the flow is password, knows the token URL, and lists the scopes as checkboxes because the API declared them.

Available authorizations ✕

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.

API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Sign in with one of the demo accounts and tick the scopes to request. The token is sent as an Authorization header.

Token URL: `auth/token`

Flow: `password`

username:

password:

Client credentials location:

Authorization header ▾

client_id:

client_secret:

Scopes: [select all](#) [select none](#)

Figure 29: The Authorize dialog, showing the password flow and the token URL

Filled in as manager, with two of the three scopes ticked:

Available authorizations

manager

password:

.....

Client credentials location:

Authorization header

client_id:

client_secret:

Scopes: [select all](#) [select none](#)

☒

reports:read
Read expense reports

☒

reports:write
Create and update expense reports

☐

reports:delete
Delete expense reports

Authorize

Close

Figure 30: Requesting reports:read and reports:write

Pressing Authorize performs the token exchange against /auth/token. The dialog then shows the session as authorised:

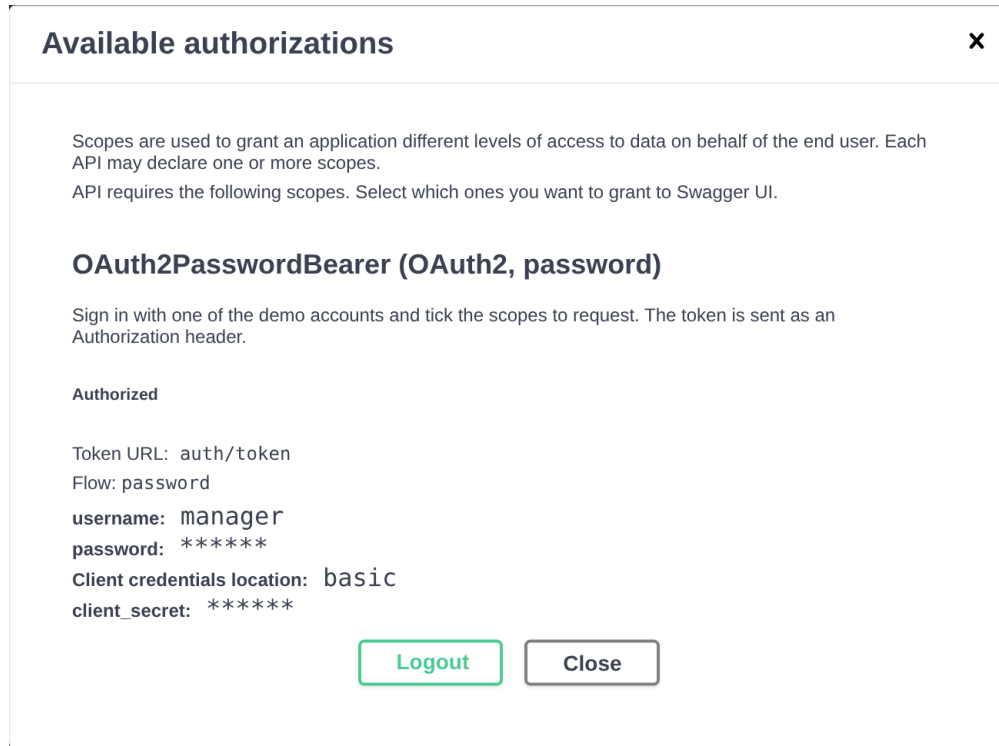


Figure 31: Authorized. Swagger UI now holds a token and sends it with every call.

The token endpoint as documented:

POST

/auth/token

Exchange username and password for tokens

The OAuth2 password grant. Send **username** , **password** and optionally **scope** as form fields. Scopes the account is not entitled to are refused rather than silently dropped.

Parameters

Try it out

No parameters

Request body **required**

application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: ^password\$

username **required**

string

password **required**

string(\$password)

scope

string

client_id

string | (string | null)

client_secret

string | (string | null)(\$password)

Responses

Figure 32: POST /auth/token, with its form fields and every response it can give

With the session authorised, Try it out sends the token. This response came back from the running API:

[illegible]

Figure 34: A live 403 from /docs: authenticated, not authorised

The generated schemas, including the Error shape every failure uses:

Schemas			^
AccessToken	>	Expand all	object
Body_issue_token_auth_token_post	>	Expand all	object
Error	>	Expand all	object
HTTPValidationError	>	Expand all	object
Identity	>	Expand all	object
NewReport	>	Expand all	object
RateLimited	>	Expand all	object
Report	>	Expand all	object
ReportList	>	Expand all	object
Summary	>	Expand all	object
TokenPair	>	Expand all	object
ValidationError	>	Expand all	object

Figure 35: The generated schemas

The same document also renders as ReDoc:

Search...

auth>

reports>

service>

Secure Expense Reports API (1.0.0)

Download OpenAPI specification: [Download](#)

An expense reports API, used here to demonstrate how the endpoints are secured rather than for the reports themselves.

Signing in

`POST /auth/token` takes a username and password as form fields, the OAuth2 password grant, and returns a short lived **access token** plus a longer lived **refresh token**. Send the access token as `Authorization: Bearer <token>`.

Press **Authorize** to sign in from this page. The scopes are checkboxes: a token only carries what was asked for, so requesting fewer scopes gives a token that can do less.

Accounts

Username	Password	Scopes
analyst	analyst-password	reports:read
manager	manager-password	reports:read, reports:write
admin	admin-password	all three

Rate limits

`5/minute` on the token endpoint, counted per address, because that is the endpoint a password guesser would use. `10/minute` on writes, `20/minute` on the summary and `60/minute` everywhere else, counted per token. Going over returns `429` with a `Retry-After` header.

auth

Getting and refreshing tokens.

Exchange username and password for tokens

The OAuth2 password grant. Send `username`, `password` and optionally `scope` as form fields. Scopes the account is not entitled to are refused rather than silently dropped.

REQUEST BODY SCHEMA: `application/x-www-form-urlencoded`

required

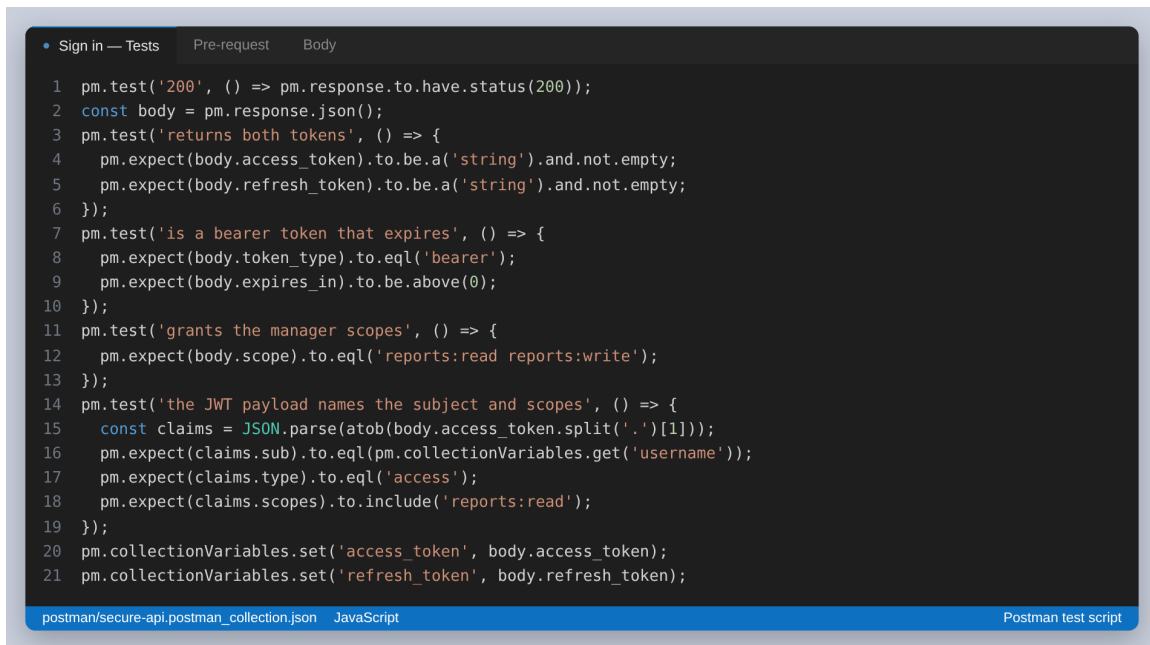
grant_type	Grant Type (string) or Grant Type (null) (Grant Type)
username	string (Username)
password	string <password> (Password)
scope	string (Scope) Default: ""
client_id	Client Id (string) or Client Id (null) (Client Id)
client_secret	Client Secret (string) or Client Secret (null) encase words (Client Secret)

Figure 36: The API at /redoc

Step 4, continued: the Postman collection

The collection is at `postman/secure-api.postman_collection.json`, in Postman's v2.1 format, with 16 requests in three folders: Auth, Reports and Rejected.

The piece that makes it pleasant to use is the test script on the sign-in request. It saves both tokens into collection variables, and the collection's own auth is set to send the access token as a bearer, so every request after sign-in is authorised without anything being pasted anywhere. It also decodes the payload and checks the claims:



```
1 pm.test('200', () => pm.response.to.have.status(200));
2 const body = pm.response.json();
3 pm.test('returns both tokens', () => {
4   pm.expect(body.access_token).to.be.a('string').and.not.empty;
5   pm.expect(body.refresh_token).to.be.a('string').and.not.empty;
6 });
7 pm.test('is a bearer token that expires', () => {
8   pm.expect(body.token_type).to.eql('bearer');
9   pm.expect(body.expires_in).to.be.above(0);
10 });
11 pm.test('grants the manager scopes', () => {
12   pm.expect(body.scope).to.eql('reports:read reports:write');
13 });
14 pm.test('the JWT payload names the subject and scopes', () => {
15   const claims = JSON.parse(atob(body.access_token.split('.')[1]));
16   pm.expect(claims.sub).to.eql(pm.collectionVariables.get('username'));
17   pm.expect(claims.type).to.eql('access');
18   pm.expect(claims.scopes).to.include('reports:read');
19 });
20 pm.collectionVariables.set('access_token', body.access_token);
21 pm.collectionVariables.set('refresh_token', body.refresh_token);
```

postman/secure-api.postman_collection.json JavaScript Postman test script

Figure 37: The test script on the Sign in request

Every request carries assertions, which means the collection is not only documentation but a check on the API. The Rejected folder asserts the refusals: that no token gives 401, that a tampered token names the signature, that a missing scope gives 403 and not 401, and that a wrong password gives a message which does not reveal whether the account exists.

Newman is Postman's command line runner and reads the same file, so the collection can be run without the app:

```

> _ Windows PowerShell - newman run

PS C:\Users\John\secure-api> newman run postman\secure-api.postman_collection.json
newman

Secure Expense Reports API

□ Auth
↳ Health (no token)
  GET http://127.0.0.1:8000/health [200 OK, 262B, 21ms]
  ✓ 200 without a token
  ✓ reports the auth scheme

↳ Sign in
  POST http://127.0.0.1:8000/auth/token [200 OK, 897B, 234ms]
  ✓ 200
  ✓ returns both tokens
  ✓ is a bearer token that expires
  ✓ grants the manager scopes
  [
  | `Using "atob" is deprecated. Use "require('atob') in s
  | tead.`
  |
  |
  ✓ the JWT payload names the subject and scopes

↳ Who am I
  GET http://127.0.0.1:8000/auth/me [200 OK, 373B, 3ms]
  ✓ 200
  ✓ is the account that signed in
  ✓ carries the expected scopes
  ✓ the token has an id and an expiry

↳ Refresh the access token
  POST http://127.0.0.1:8000/auth/refresh [200 OK, 569B, 2ms]
  ✓ 200
  ✓ returns a new access token

... the Reports and Rejected folders follow

```

Figure 38: newman run, showing the Auth folder and its assertions

The totals for the whole run:

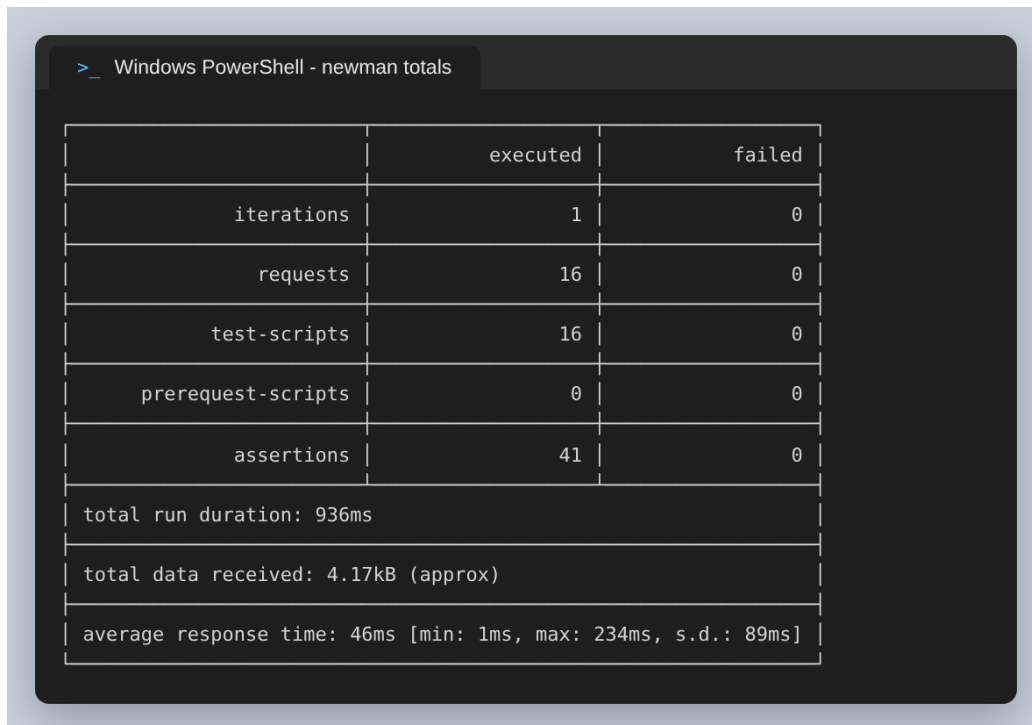


Figure 39: 16 requests, 41 assertions, nothing failed

One assertion did fail the first time I ran this, and it was my own mistake rather than the API's: the collection saved the new report's id as a number and then compared it against a string. Worth mentioning because it is exactly the sort of thing that makes a collection worse than nothing if it is written and never executed.

Step 5: The practical use case

The client is a single page served by the API itself at /app. It signs in, holds the token, and calls the same endpoints as everything above. It is the practical use case the assignment asks for, and it is also where the security decisions become visible to a person.

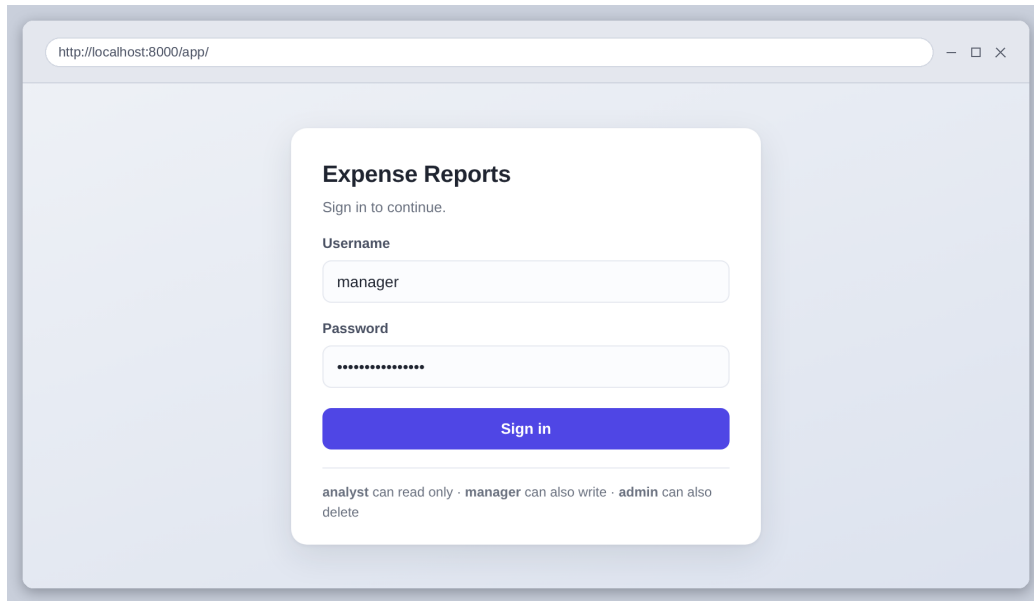


Figure 40: The sign-in screen. This posts to `/auth/token`, the same endpoint curl used.

Signed in as manager. The scopes the token carries are shown next to the account name, the totals come from `/reports/summary` and the table from `/reports`, each with the token attached:

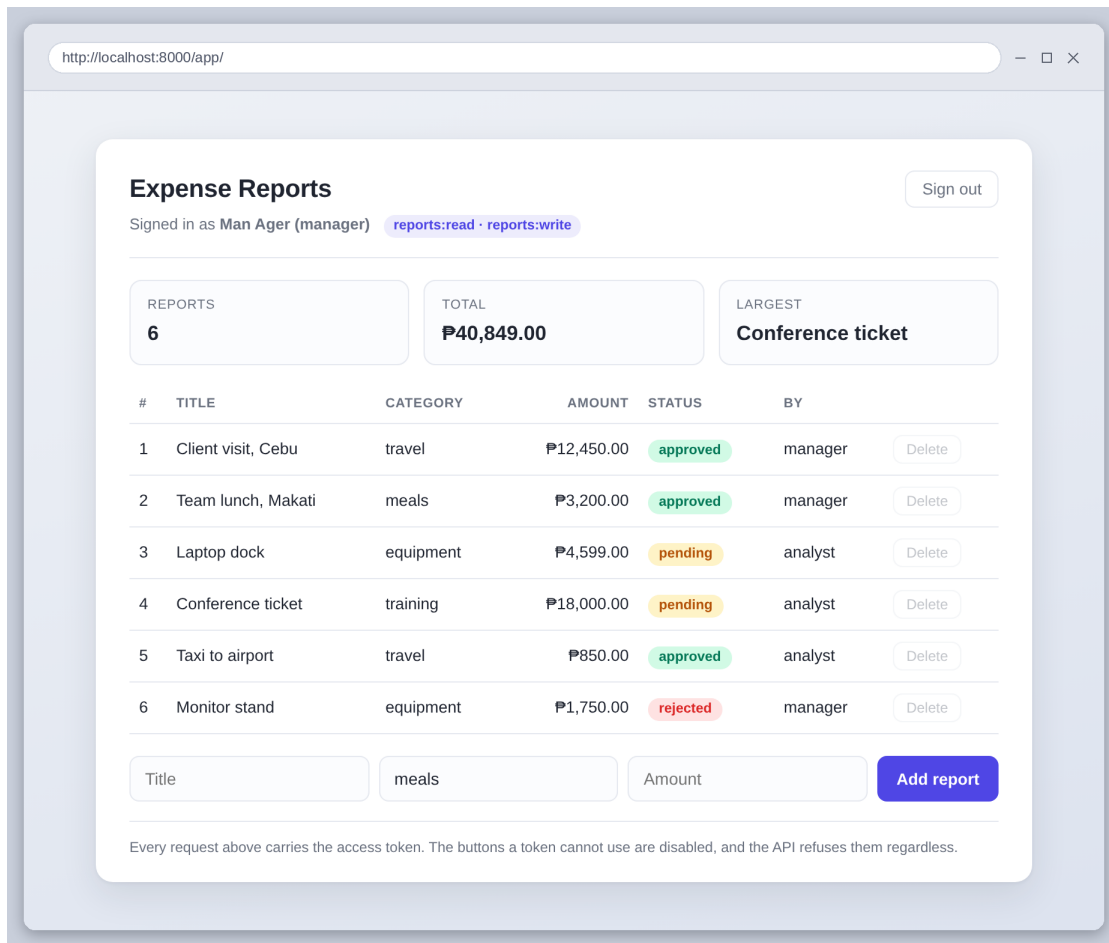


Figure 41: The client, signed in as manager

Adding a report, which needs `reports:write`:

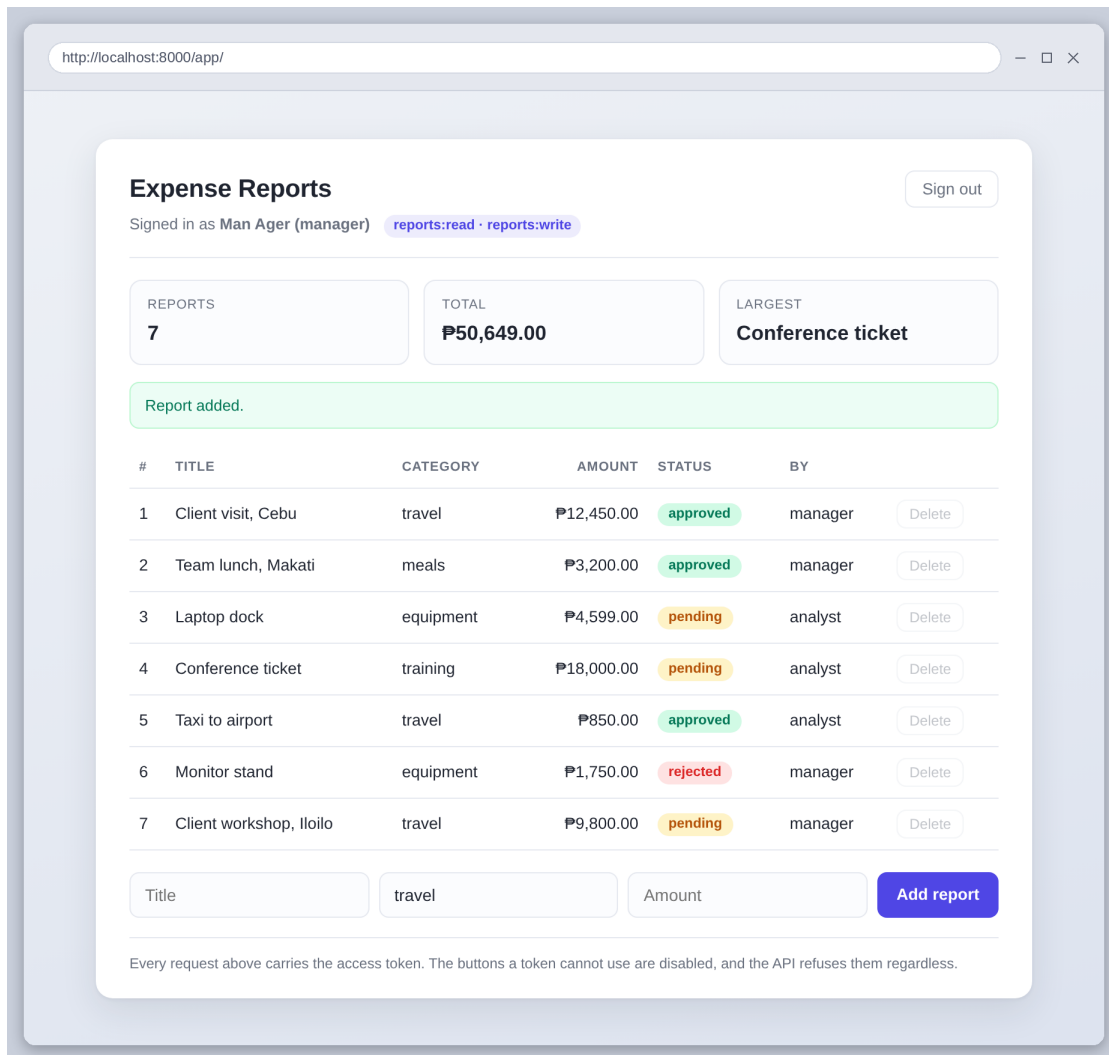


Figure 42: A report created through the client

The delete buttons are disabled, because this token does not carry `reports:delete`. That is a courtesy to the user, not a security measure, and the distinction matters. To show why, I enabled a delete button in the browser and pressed it, which is exactly what a modified client would do. The API refused it:

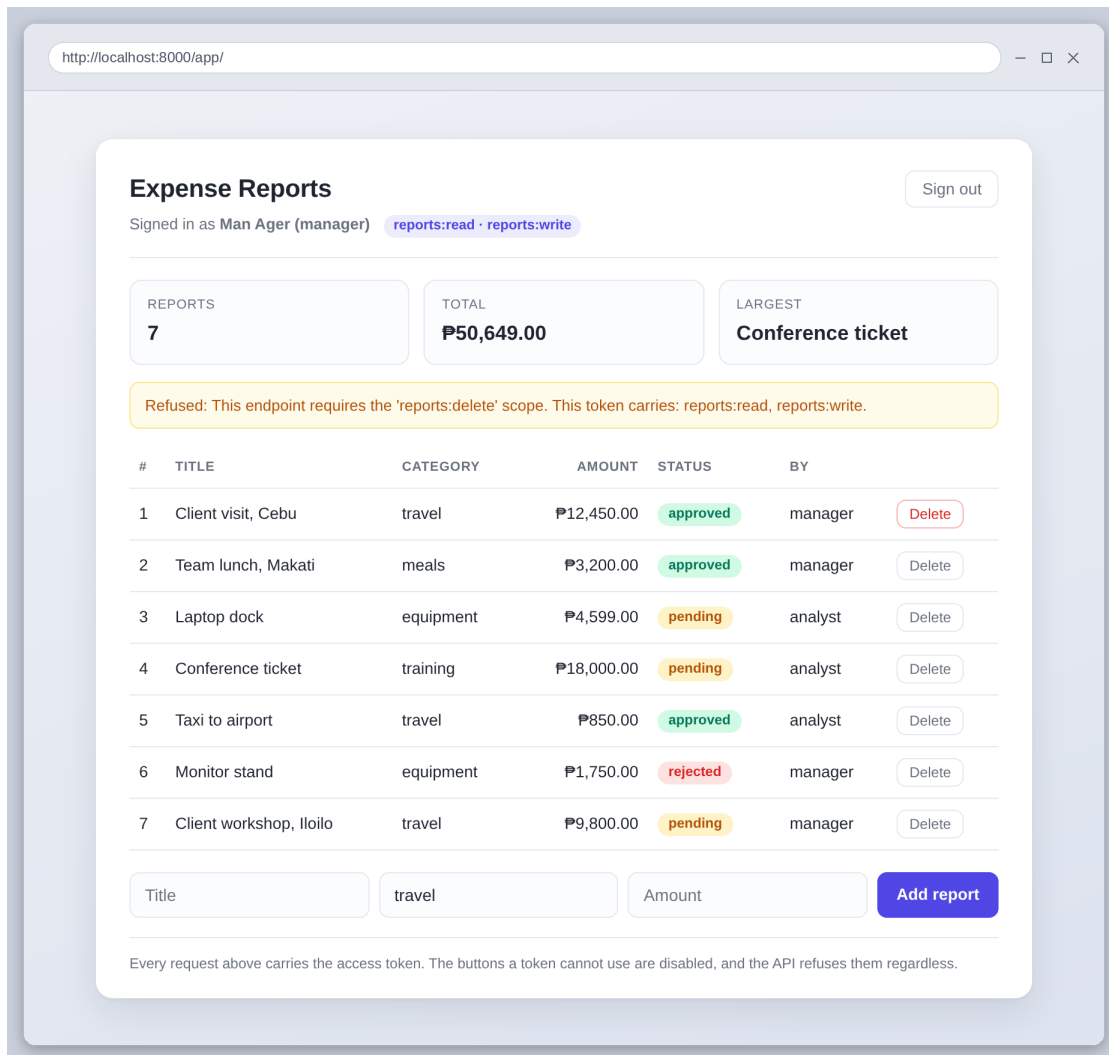


Figure 43: The API refusing a delete after the disabled button was re-enabled in the browser. The message is the API's own.

Signed in as analyst instead, the add form is not rendered at all, because that token cannot write:

http://localhost:8000/app/

Expense Reports

Sign out

Signed in as Ana Lyst (analyst)

reports:read

REPORTS
7

TOTAL
₱50,649.00

LARGEST
Conference ticket

Refused: This endpoint requires the 'reports:delete' scope. This token carries: reports:read, reports:write.

#	TITLE	CATEGORY	AMOUNT	STATUS	BY	
1	Client visit, Cebu	travel	₱12,450.00	approved	manager	Delete
2	Team lunch, Makati	meals	₱3,200.00	approved	manager	Delete
3	Laptop dock	equipment	₱4,599.00	pending	analyst	Delete
4	Conference ticket	training	₱18,000.00	pending	analyst	Delete
5	Taxi to airport	travel	₱850.00	approved	analyst	Delete
6	Monitor stand	equipment	₱1,750.00	rejected	manager	Delete
7	Client workshop, Iloilo	travel	₱9,800.00	pending	manager	Delete

Title

travel

Amount

Add report

Every request above carries the access token. The buttons a token cannot use are disabled, and the API refuses them regardless.

Figure 44: The same client with a read-only token

And the login rate limit as a user meets it, after six wrong passwords:

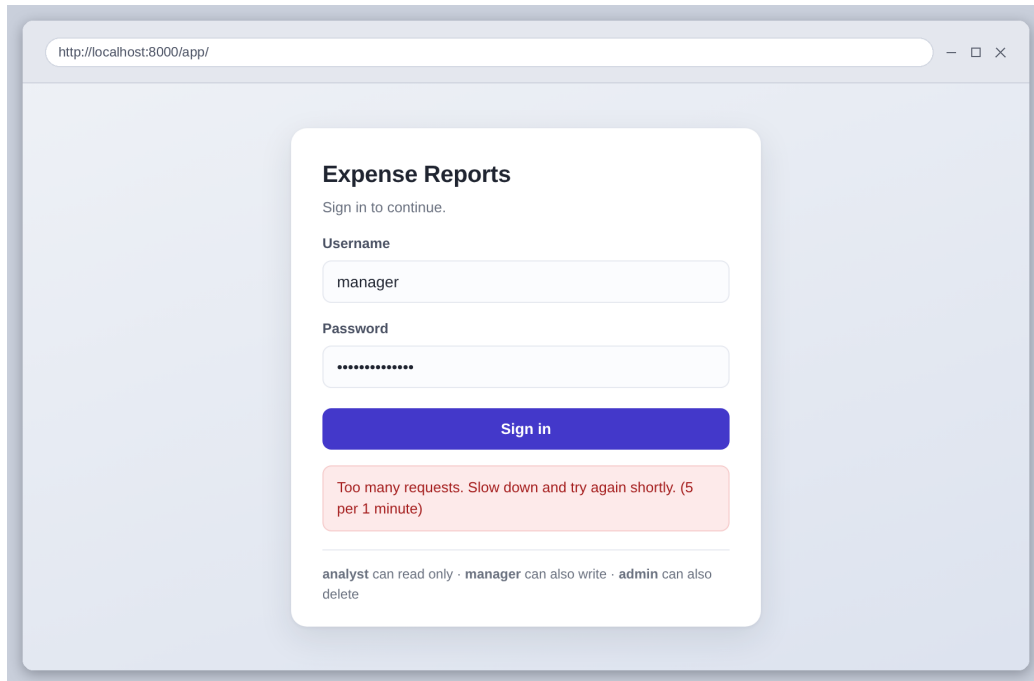


Figure 45: The 429 surfaced in the client, with the limit that was hit

Three things in the client are deliberate. The token is kept in a variable and not in localStorage, because anything in localStorage can be read by any script on the page, so one injected script would be enough to steal it. A 401 sends the user back to the sign-in screen, since a token that is expired or not trusted cannot be recovered from. And a 403 shows the API's own message rather than a generic failure, so the user is told what permission they are missing.

Tests

49 tests, in three files: 23 for authentication, 18 for scopes and 8 for the rate limits.

```
>_ Windows PowerShell - pytest

PS C:\Users\John\secure-api> python -m pytest tests/ -q
..... [100%]
49 passed in 15.32s

tests/test_auth.py      23 passed
tests/test_scopes.py    18 passed
tests/test_limits.py     8 passed

PS C:\Users\John\secure-api> python -m pytest tests/test_limits.py -v
test_the_sixth_login_attempt_in_a_minute_is_refused PASSED [ 12%]
test_the_429_says_what_the_limit_was_and_when_to_retry PASSED [ 25%]
test_failed_logins_count_towards_the_limit PASSED [ 37%]
test_the_login_limit_does_not_block_reading PASSED [ 50%]
test_writes_have_a_tighter_limit_than_reads PASSED [ 62%]
test_reads_are_still_allowed_after_writes_are_limited PASSED [ 75%]
test_two_tokens_get_separate_allowances PASSED [ 87%]
test_the_summary_limit_is_tighter_than_the_default PASSED [100%]
===== 8 passed in 5.95s =====
```

Figure 46: The test run, and the rate limit tests by name

The ones I would keep if I could keep only a few are the attacks. One edits the scopes inside a token's payload to add reports:delete and confirms the request is still refused, which is the test that proves the signature is doing its job. One signs a well-formed token with a different secret. One presents a refresh token as an access token. One checks that a wrong password and an unknown username produce byte for byte the same response.

The rate limit tests needed a fixture I did not expect to write. The limiter counts across requests, which is the point of it, so one test's traffic was spending the next test's allowance and unrelated tests were failing with 429s. It is switched off by default and turned on only for the tests that are about limiting.

The same problem bit the screenshot script. Its own setup was signing in often enough to trip the five-per-minute login limit, so it started collecting 429s it had not asked for. It now restarts the server between groups of captures, since the counters live in the process.

What I would change

The tokens cannot be revoked. Every token carries a jti precisely so that a specific one could be blacklisted, but nothing reads it yet, so a leaked access token stays usable for its full 15 minutes and a refresh token for a day. A store of revoked ids checked on each request is the first thing I would add.

The secret has a fallback default so the project runs after a clone, which is convenient and wrong. It should refuse to start without SECRET_KEY set in the environment. The accounts are in a module for the same reason and would be a database.

The rate limit counters are in memory, so they are per process. Two workers would give an attacker twice the allowance, and a restart clears the count entirely. Flask-Limiter and SlowAPI both support Redis for this, which is what a deployment would use.

What I got most out of was the difference between 401 and 403. Before this I would have treated them as roughly interchangeable. Building the scope dependency made it concrete: they are answers to two different questions, and a client can only respond sensibly, by signing in again or by giving up, if the API is precise about which one it is asking.